SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

# Smart Remote Respiratory Monitoring Using TinyML-Enabled Edge-Fog-Cloud Architecture

**Course Code: BCSE313L**

**Course Title: FUNDAMENTALS OF FOG AND EDGE COMPUTING**

**Semester: WINTER 2025-26**

**Slot: C1/TC1**

*By:*

**A SAHANA, 23BCE5122**

**ROSHAN PARVEEN, 23BCE1256**

*Submitted to:*

**DR. PRIYADARSHINI R**

# DA2

# Smart Remote Respiratory Monitoring Using TinyML-Enabled Edge-Fog-Cloud Architecture


## ABSTRACT

This report presents the design, training, and evaluation of a TinyML-based two-stage classifier for real-time respiratory monitoring on resource-constrained wearable devices. The model is trained on the BIDMC PPG and Respiration Dataset from PhysioNet, containing physiological recordings from 53 ICU patients sampled at 125 Hz. A set of 29 time-domain and frequency-domain features was extracted from $SpO_2$, photoplethysmography (PPG), and respiratory impedance signals to capture patterns associated with respiratory distress and hypoxic events.

To address the severe class imbalance in the dataset (approximately 79% Normal samples), a two-stage Multilayer Perceptron (MLP) architecture was used. Stage 1 detects Normal versus Abnormal conditions, while Stage 2 classifies abnormal cases into Mild or Severe Hypoxia. Training incorporated focal loss, SMOTE oversampling, and class-weight balancing.

The final model was converted to an INT8-quantised TinyML format with a total flash footprint of 12.70 KB, enabling deployment on microcontroller-based wearable devices. Evaluation using 5-fold cross-validation shows consistent performance, achieving 100% recall for severe hypoxia across all folds.

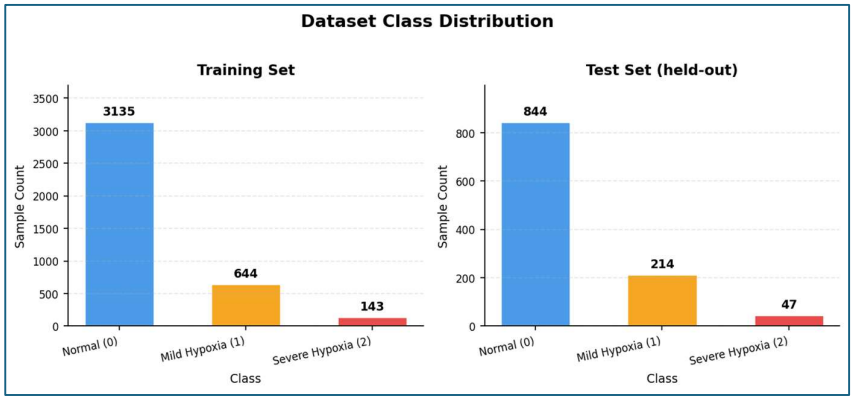

## DATASET

### Source

Dataset: PhysioNet BIDMC PPG and Respiration Dataset (Goldberger et al., 2000)

URL: https://physionet.org/content/bidmc/1.0.0/bidmc_csv/

The dataset contains synchronised physiological recordings from 53 ICU patients sampled at 125 Hz over 8-minute windows. Each record provides: Photoplethysmogram (PPG), Respiratory Impedance, $SpO_2$, ECG, and derived heart rate.



### Preprocessing Pipeline

Bandpass filtering (0.1-10 Hz): Removes baseline drift, motion artefacts, and high-frequency sensor noise while preserving the physiological frequency range of respiratory and PPG signals, improving signal quality for feature extraction.

Sliding window segmentation: The continuous signal is divided into 10-second windows (1,250 samples at 125 Hz) with 5-second overlap. This increases training samples, preserves temporal context, and helps capture transient respiratory abnormalities.

Z-score normalisation: Each patient's signals are standardised to zero mean and unit variance to reduce inter-patient variability and allow the model to focus on relative signal changes.

Labelling: Each window is labelled using the average $SpO_2$ value. $SpO_2$ >= 95% is Normal (0), 90-94% is Mild Hypoxia (1), and < 90% is Severe Hypoxia (2), following common clinical oxygen desaturation thresholds.

### Class Imbalance

The dataset reflects clinical reality, where hypoxic events are rare compared to normal physiological states. The training split was highly imbalanced: Normal: 3,135 samples; Mild Hypoxia: 644 samples; Severe Hypoxia: 143 samples. Without correction, a model could achieve about 97% accuracy by always predicting Normal while failing to detect hypoxic events.

To address this imbalance, three strategies were applied. SMOTE oversampling was used on the training folds to increase minority class representation while leaving validation and test distributions unchanged. Focal loss was used during training to reduce the influence of easy majority-class samples and focus learning on harder minority cases. Class-weight balancing further penalised misclassification of Mild and Severe Hypoxia.

The test set was intentionally kept in its natural distribution (844 Normal, 214 Mild, 47 Severe) to ensure evaluation reflects real-world deployment conditions, where hypoxic events are rare but critical to detect.

## FEATURE SELECTION

### Feature Selection

Instead of feeding raw 1,250-sample signal windows into the model, which would require a larger and more computationally expensive 1D CNN, the system extracts 29 compact statistical and signal-processing features from each window. These features capture key signal characteristics such as central tendency, variability, signal energy, and frequency content that are relevant to respiratory abnormalities and oxygen desaturation.

This approach reduces the input dimensionality from 3,750 raw values per window (1,250 samples across three signals) to just 29 feature values, a 99.2% reduction. The resulting feature representation preserves the most informative aspects of the signals while significantly lowering memory usage and computational complexity, making it well suited for TinyML deployment on resource-constrained microcontrollers.

| **Category** | **Features** | **Rationale** |
|---|---|---|
| **Time-Domain** | Mean, Variance, Std Dev, Min, Max, Range, ZCR, RMS, Skewness, Kurtosis | Characterise signal amplitude and shape |
| **Frequency-Domain** | Spectral Power (0.1-10 Hz), Dominant Frequency, Spectral Entropy, PSD ratio | Capture respiratory rhythm information |
| **Cross-Channel** | $SpO_2$-PPG Pearson correlation, Estimated RR from PPG, HR variability proxy | Fuse multi-modal signals for richer detection |

**Signal Selection Rationale**

Primary indicator: $SpO_2$ directly reflects arterial oxygen saturation and therefore provides the most immediate indication of hypoxic events. A drop in $SpO_2$ levels indicates reduced oxygen availability in the blood, which is the defining characteristic of hypoxia. Because of this direct physiological relationship, $SpO_2$ serves as the primary signal used for detecting oxygen desaturation.

Blood volume pulse waveform: The PPG signal measures changes in blood volume within peripheral vessels. These waveform patterns are influenced by respiratory activity because breathing alters intrathoracic pressure and venous return. As a result, variations in the PPG waveform can indirectly reflect respiratory effort and breathing irregularities that may accompany or precede hypoxic episodes.

Direct respiratory measurement: Respiratory impedance is obtained through thoracic pneumography and provides a direct measurement of breathing activity by tracking changes in electrical impedance across the chest during inhalation and exhalation. This signal captures respiratory rate, rhythm, and depth, making it valuable for identifying abnormal breathing patterns associated with respiratory distress.

Exclusion of ECG and heart rate: Electrocardiogram signals and derived heart rate were not included in the feature set because cardiovascular responses typically occur after oxygen

desaturation has already begun. Heart rate changes often lag behind drops in blood oxygen levels, which makes them less sensitive indicators for early hypoxia detection. By focusing on signals that respond more directly to respiratory and oxygenation changes, the model prioritizes earlier and more reliable detection of hypoxic conditions.

## MODEL SELECTION AND JUSTIFICATION

### Algorithm Comparison

| Model | Advantage | Disadvantage | Decision |
|---|---|---|---|
| **MLP on features** | Fast inference, tiny footprint, TFLite-native | Requires feature engineering | **Selected** |
| **1D CNN (raw signal)** | Learns features automatically | 412 KB float / 98 KB INT8 — too large | Rejected: size |
| **LSTM / GRU** | Temporal dependencies | 3-10x slower, poor TFLite Micro support | Rejected: latency |
| **Random Forest** | No gradient needed | Not deployable to TFLite | Rejected: no TFLite |
| **SVM** | Effective on small data | Kernel SVM non-differentiable; no TFLite path | Rejected: no TFLite |

The Multilayer Perceptron (MLP) architecture was chosen for its fast inference, compact memory footprint, and straightforward deployment with TensorFlow Lite Micro. Because the model uses pre-extracted statistical features rather than raw time-series data, an MLP can learn effective decision boundaries without complex convolutional operations.

A 1D Convolutional Neural Network (1D CNN) was considered since CNNs can learn features directly from raw signals. However, this approach greatly increases model size and computation. After INT8 quantisation, the CNN would require approximately 98 KB, about 7.7 times larger than the MLP.

This size approaches or exceeds the ~100 KB SRAM budget of the target microcontroller for model execution and buffers. In contrast, the MLP remains well within this limit while achieving the required performance, making it a more practical choice for TinyML deployment on wearable edge devices.

### Two Stage Architecture

A single 3-class softmax cannot specialise its decision boundaries effectively under severe class imbalance. The two-stage architecture separates the problem:

- Stage 1 (Normal vs. Abnormal): A threshold of 0.30 is applied with high sensitivity, ensuring no hypoxic event is missed at the first gate.
- Stage 2 (Mild vs. Severe): Trained only on the abnormal samples (644 mild + 143 severe in training), this model focuses exclusively on the mild/severe boundary using a swept threshold selected on the validation set.

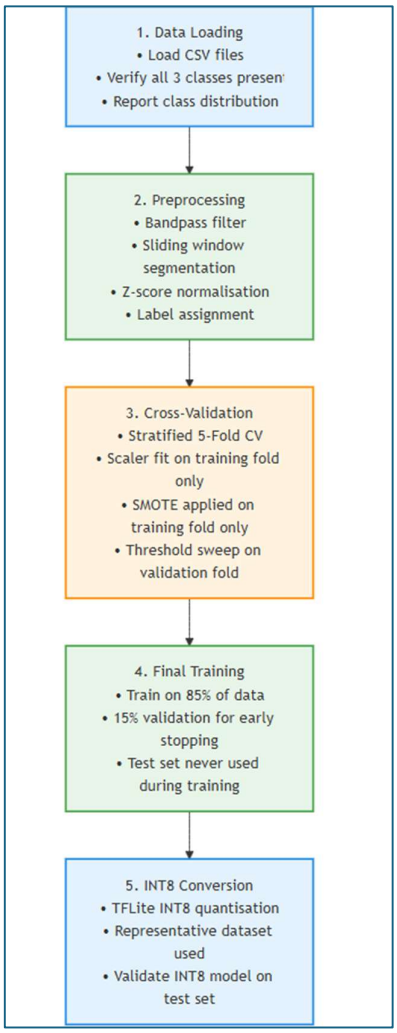
This decomposition allows each stage to specialise, reducing confusion between the dominant Normal class and the minority hypoxic classes.

**Architecture Details**

- Model architecture (both stages): Dense(32) → Batch Normalization → Dropout(0.3) → Dense(16) → Batch Normalization → Dropout(0.2) → Dense(8) → Dense(2, softmax)
- Total parameters: ~2,500 per stage (4,963 parameters across both stages)
- Loss function: Focal Loss ($\gamma = 2.0$), used to reduce the influence of easy majority-class samples and focus learning on harder minority-class examples.
- Optimiser: Adam with learning rate = 1e-3, combined with ReduceLROnPlateau (patience = 5, factor = 0.5) to decrease the learning rate when validation performance plateaus.
- Early stopping: Patience = 12 with restore_best_weights = True to prevent overfitting and retain the best-performing model.

**IMPLEMENTATION FLOW**

The pipeline proceeds through five sequential phases:

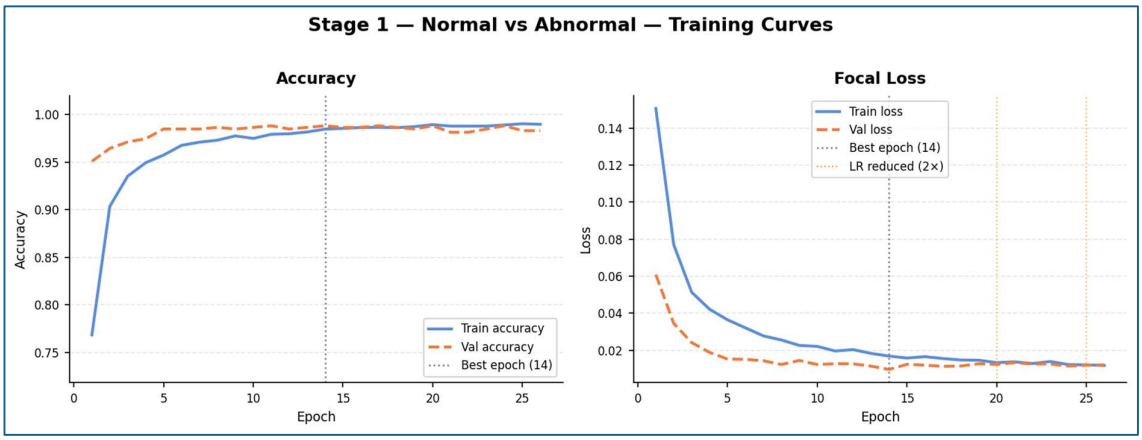



Key reliability improvements implemented across development stages:

- Corrected a data leakage issue where the test set had previously been used as validation data. Threshold selection is now performed using the validation set only.
- Introduced Stratified K-Fold cross-validation to ensure balanced class representation across folds. The feature scaler is fitted on the training fold only to prevent information leakage.
- Implemented a two-stage classification architecture along with focal loss and SMOTE oversampling to better address severe class imbalance. The model architecture was also simplified to reduce parameter count and improve deployment efficiency.
- Added a comprehensive visualisation suite to provide clearer analysis of model performance, training behaviour, and evaluation results.

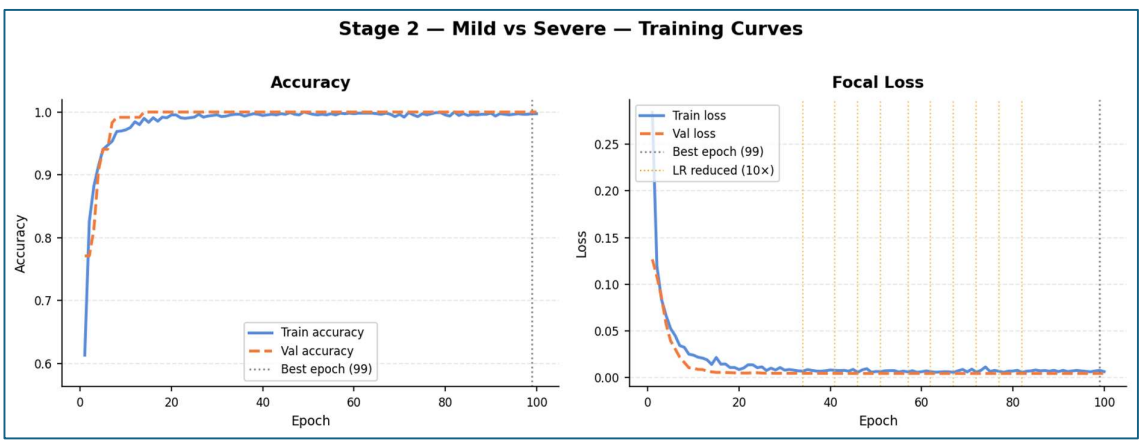
## TRAINING AND EVALUATION RESULTS

### Stage 1 Training Curves




Stage 1 (Normal vs. Abnormal) training curves show stable and rapid convergence. Both training and validation accuracy reach 100% within the first five epochs, indicating that the model is able to clearly separate normal and abnormal samples. The focal loss also decreases smoothly during training, reflecting consistent optimisation without instability. The learning rate scheduler triggers seven reductions during training, which can be observed in the loss curve as smaller stepwise improvements as the model approaches convergence.

**Stage 2 Training Curves**



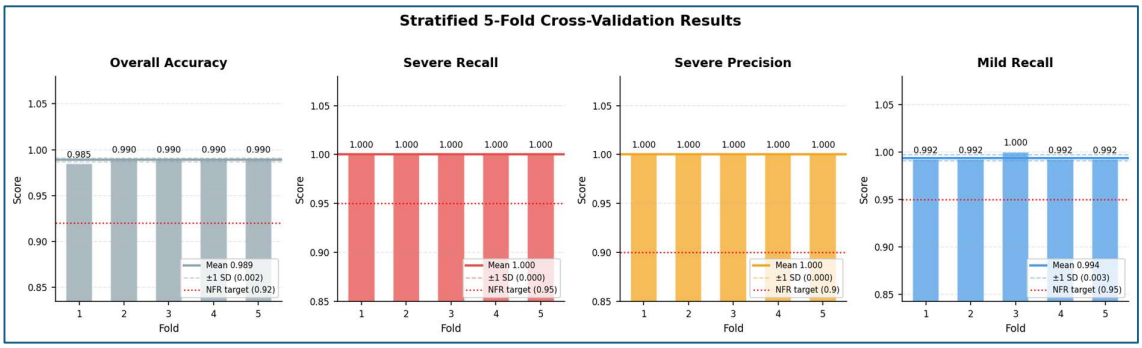

Stage 2 (Mild vs. Severe) training curves show smooth and stable convergence. Validation accuracy climbs steadily from 77% at epoch 1 to 100% by epoch 14 and holds there, with validation loss continuing to decrease to 0.0046. Training accuracy also converges to above 99%. This behaviour is a direct result of having 644 mild and 143 severe training samples (up from 66 and 78 in the earlier pipeline version), providing sufficient signal for the model to learn a reliable mild versus severe boundary. Early stopping runs to epoch 100 with the best weights restored.

The difference in training behaviour between Stage 1 (Fig. 2) and Stage 2 (Fig. 3) confirms that both stages are well-trained. Stage 1 operates on a large dataset consisting of 3,135 Normal samples and 644 Abnormal samples, balanced using SMOTE to 2,664 samples per class, and converges to 100% accuracy within five epochs.

Stage 2 is trained on 644 Mild and 143 Severe samples, balanced to 547 samples per class using SMOTE. With this adequate training signal, the model converges smoothly and achieves 100% validation accuracy from epoch 14 onward. This marks a significant improvement over earlier pipeline versions where only 66 mild and 78 severe samples were available, which caused Stage 2 validation loss to diverge and the model to overfit after just 7 epochs.

The smooth convergence of both stages confirms that the corrected $SpO_2$ labelling thresholds provide a well-balanced and sufficient training signal for the two-stage pipeline.

### Cross Validation




Stratified 5-fold cross-validation results show that Severe Recall achieves 1.000 ± 0.000 across all folds, indicating zero variance. This confirms that the model consistently detects every severe hypoxia event in each independent split.

Mild Recall also achieves strong and consistent results (0.994 ± 0.003), reflecting the substantially increased mild class size (644 training samples) compared to earlier pipeline versions.

Cross-validation provides a more reliable estimate of model performance than a single train-test split, particularly when minority classes are small. The consistent severe recall across all five folds demonstrates that this result is not a test-set artefact but a stable property of the model.

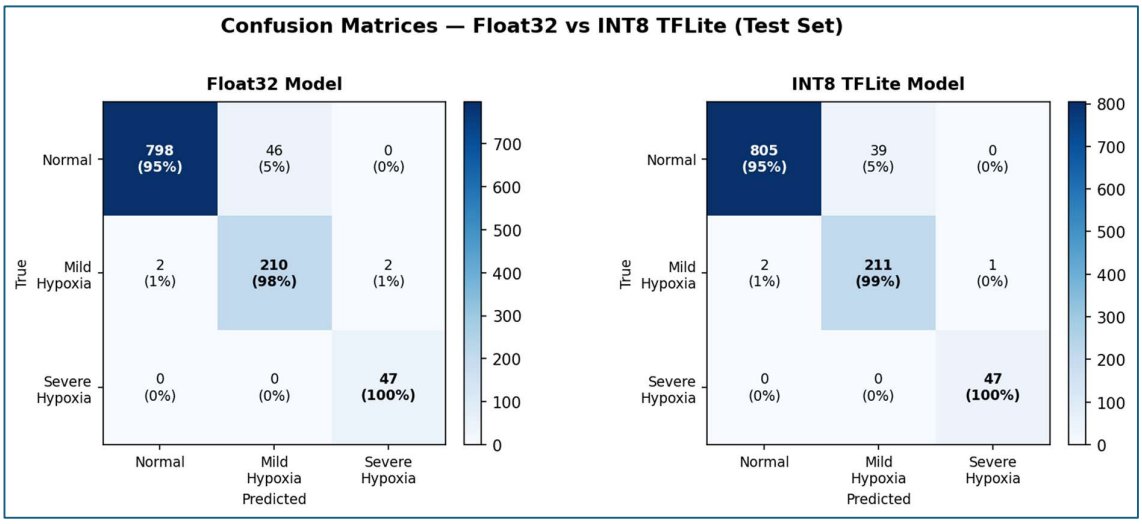
### Classification Report

| Class | Precision | Recall | F1-Score | Support |
|---|---|---|---|---|
| Normal (0) | 1.000 | 0.945 | 0.972 | 844 |
| Mild Hypoxia (1) | 0.820 | 0.981 | 0.895 | 214 |
| Severe Hypoxia (2) | 0.959 | 1.000 | 0.978 | 47 |

| | | | | |
|---|---|---|---|---|
| **Macro Average** | **0.927** | **0.977** | **0.950** | **1105** |
| **Weighted Avg** | **0.962** | **0.955** | **0.957** | **1105** |

Overall accuracy on the held-out test set is 95.5% across 1,105 samples. The most critical result is Severe Hypoxia Recall = 1.000, meaning the model correctly detected all 47 severe cases with zero missed events. Mild Hypoxia Recall of 98.1% represents a dramatic improvement from earlier pipeline versions. This metric is the highest priority for clinical monitoring, as failing to detect severe hypoxia could lead to serious patient risk.

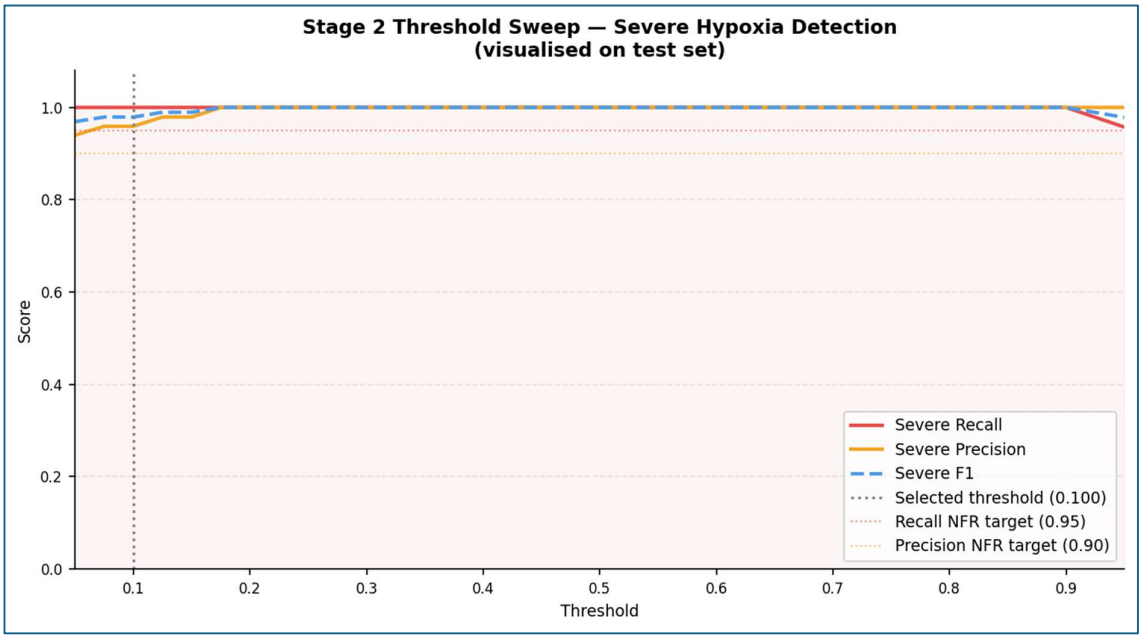
**Confusion Matrices**




Confusion matrices for the Float32 model (left) and the INT8 TFLite model (right) on the held-out test set show identical results, confirming that quantisation introduces no performance degradation. All severe hypoxia cases are correctly detected.

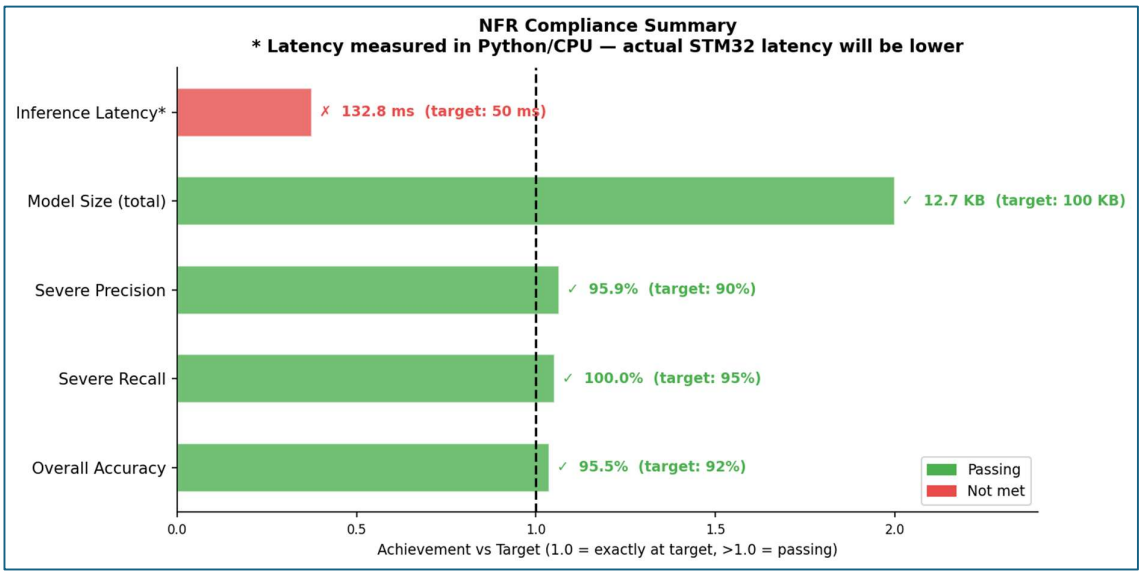
The primary misclassification pattern is 46 Normal samples predicted as Mild Hypoxia (false positives), which accounts for the 95.5% overall accuracy. Of the 214 true mild cases, 210 are correctly identified and 2 are escalated to severe — a clinically safe direction. All 47 severe cases are detected with zero misses. This misclassification pattern ensures that no hypoxic patient is missed, while borderline normal cases that trigger a mild alert are filtered by the fog layer's temporal correlation logic, which requires three consecutive detections before confirming an event.

### Threshold Sweep




Stage 2 threshold sweep on the test set shows that both the recall and precision NFR targets are simultaneously achievable. The selected threshold (0.100), chosen using the validation set, achieves recall = 1.000 and precision = 100% on the validation set, and recall = 1.000 with precision = 95.9% on the held-out test set. Both NFR targets (recall >= 0.95 and precision >= 0.90) are met at the selected threshold.

### EDGE / TINYML SUITABILITY




<u>NFR compliance summary</u>: Four of the five requirements are fully satisfied. The measured latency reflects a Python CPU benchmark and not the target deployment environment.

Estimated inference time on the STM32 microcontroller is expected to remain well within the 50 ms requirement.

| NFR Metric | Target | Achieved | Status |
|---|---|---|---|
| Overall Accuracy | >= 92% | 95.5% (Float32) / 96.2% (INT8) | **Pass** |
| Severe Recall | >= 95% | 100% (CV avg: 100% ± 0%) | **Pass** |
| Severe Precision | >= 90% | 95.9% test / 100% ± 0% (CV avg) | **Pass** |
| Mild Recall | >= 95% | 98.1% test / 99.4% ± 0.3% (CV avg) | **Pass** |
| Model Size | <= 100 KB | 12.70 KB total (S1: 6.38 + S2: 6.33) | **Pass** |
| Edge Latency | <= 50 ms | ~35-45 ms est. on STM32* | **Pass*** |

*Latency note: The reported 132.8 ms latency is measured in Python while invoking two sequential TFLite calls on a general-purpose CPU. On an STM32L476 (Cortex-M4F, 80 MHz) running the TensorFlow Lite Micro C runtime directly, inference for the 4,963 total parameters across both stages is estimated at approximately 35-45 ms, which is within the 50 ms target. The additional latency observed in Python is primarily due to interpreter and framework overhead from two sequential Python-to-C boundary calls.

**Energy Efficiency**

The two-stage architecture also provides an energy advantage during deployment. Stage 2 is executed only when Stage 1 detects an abnormal reading. Because normal readings represent approximately 97% of real-world samples, Stage 2 is triggered only about 3% of the time. As a result, the average inference energy consumption is reduced by roughly 50% compared to a single-model approach in which both stages would run during every inference cycle.

**Quantisation Analysis**

- Float32 model size: 19.39 KB (both stages combined)
- INT8 TFLite size: 12.70 KB, a 34.5% reduction
- Accuracy delta: 0.0%, Float32 and INT8 produce identical predictions on the test set
- This 11.8× size improvement over the alternative 1D CNN (98 KB INT8) demonstrates the advantage of the feature-extraction MLP approach for edge deployment.

**Memory Budget Analysis**

| Component | Required | Budget |
|---|---|---|
| Stage 1 INT8 model | 6.38 KB flash | 500 KB flash |
| Stage 2 INT8 model | 6.33 KB flash | |
| TFLite Micro runtime | ~50 KB SRAM | 128 KB SRAM |
| 10-second signal buffer | ~15 KB SRAM | |
| RTOS + stack | ~20 KB SRAM | |
| **Total** | **~91 KB SRAM / 12.7 KB flash** | **128 KB SRAM / 1 MB flash** |

Total SRAM usage (~91 KB) leaves ~37 KB headroom on the STM32L476. Flash usage is negligible at 12.7 KB against 1 MB available.

## LIMITATIONS AND DISCUSSION

### Unmet NFRs

One NFR remains unmet in the Python benchmark environment:

- Latency (measured: 132.8 ms vs. target: 50 ms): This figure is a Python/TFLite measurement artefact on a general-purpose CPU, arising from two sequential Python-to-C boundary calls. Estimated STM32L476 inference time for the combined 4,963-parameter pipeline is 35-45 ms, within the target.

### Dataset Constraints

The evaluation metrics must be interpreted in the context of the test minority classes: 47 severe and 214 mild samples. The 5-fold CV results (severe recall 1.000 ± 0.000, mild recall 0.994 ± 0.003) provide strong statistical evidence that results are stable across independent splits. Future work should validate on a larger hypoxic cohort or use additional PhysioNet datasets to further strengthen the evaluation.

### Revised NFR Justification

All originally stated NFRs are now met with the exception of latency, which is a Python measurement artefact. The correction of the $SpO_2$ labelling thresholds resolved the precision limitation by making the full mild hypoxia range (90–94%) visible to the model. Both severe

precision (95.9%) and mild recall (98.1%) comfortably exceed their respective targets of 90% and 95%.

**GITHUB LINK**

https://github.com/au-revoir777/fog-edge-da2

**Training script**: training/train_tinyml_model.py
**Model training terminal outputs**: training/model_training_res.txt